

Cora GNN Oversmoothing Study — Master Reference

Depth, Oversmoothing, and Architecture on the Cora Citation Network

Kiarash Ara (lead / integration owner), with Yiheng Wu (metrics)

2026-07-08

Table of contents

How to use this document	2
1 Project at a glance	3
2 Dependency map of the material	4
3 Notation & conventions	5
4 Stage 1 — GCN fundamentals	6
4.1 4.1 Why a plain MLP fails on a graph	6
4.2 4.2 One GCN layer = aggregate to transform to nonlinearity	6
4.3 4.3 The symmetric-normalized adjacency	6
4.4 4.4 Depth = stacked layers with interleaved nonlinearities	6
4.5 4.5 Oversmoothing and its Dirichlet-energy fingerprint	7
4.6 4.6 Cora: semi-supervised, transductive	7
Stage 1 — defense-gate prompts	7
5 Stage 2 — GraphSAGE & GAT via the spectral-vs-spatial lens	8
5.1 5.1 The lens	8
5.2 5.2 GraphSAGE (Hamilton et al., 2017)	8
5.3 5.3 GAT (Velickovic et al., 2018)	8
5.4 5.4 Side by side	9
5.5 5.5 Does relaxing GCN’s fixed filter avoid oversmoothing?	9
Stage 2 — defense-gate prompts	9
6 Stage 3 — Oversmoothing metrics & the interface contract (Yiheng’s component)	10
6.1 6.1 Dirichlet energy	10
6.2 6.2 MAD — mean average distance	10
6.3 6.3 The interface contract (the shared surface — fixed once set)	10
Stage 3 — defense-gate prompts	11

7	Stage 4 — Oversmoothing mitigations	12
7.1	7.1 Residual / skip connections	12
7.2	7.2 PairNorm (Zhao & Akoglu, 2020)	12
7.3	7.3 Jumping Knowledge (Xu et al., 2018)	12
7.4	7.4 GCNII (Chen et al., 2020)	13
7.5	7.5 Predict before you run	13
	Stage 4 — defense-gate prompts	13
8	Stage 5 — Experimental spec to Claude Code handoff	14
	Stage 5 — defense-gate prompts	14
9	The interface contract (shared surface)	15
10	Repo structure & build order	16
11	Code style & report voice	17
12	Review / defense gate & walk-through	18
13	Experiment grid & what each experiment proves	19
14	Open questions & decisions pending	20
15	Glossary	21
16	References	22

How to use this document

A **standalone technical reference** for the Cora GNN project: the full theory behind GCN, GraphSAGE, GAT, the two oversmoothing metrics, and the four mitigations, plus the integration contract, repo layout, and review mechanics. It is meant to be complete on its own — every stage is written to derivation level, not left as a stub.

Sourcing. Every architecture / mitigation / metric equation here is the form given by the source paper or by PyTorch Geometric (the library we implement against), named at the point of use. Nothing is reconstructed from memory. Where a paper and PyG differ only by an algebraically equivalent rewrite, that is noted.

Stage tags (Stage 1...Stage 5) mark the *dependency order* in which we build and defend the material, not gaps in this document. Each section ends with **defense-gate prompts** — the questions to answer, attempt-first, before a component counts as done.

Rendering: `quarto render cora_gnn_master_reference.ipynb --to pdf` (needs LaTeX, e.g. `quarto install tinytex`). `execute: enabled: false` keeps the render pure-markdown; no kernel runs.

1 Project at a glance

Research question. Not “can a GNN classify Cora” (a shallow GCN already reaches ~81–83%) but: *why do deeper message-passing GNNs get worse, and which mitigations restore deep performance?* Depth sets the receptive field — how many hops of structure a node integrates — so it is the object of study, not a nuisance knob.

Task. Semi-supervised, transductive node classification on Cora: 2,708 nodes (ML papers), undirected citation edges, 1,433-dim binary bag-of-words features, 7 classes. Standard Planetoid split: **140 train / 500 val / 1,000 test**.

Common frame (message passing). Every layer updates a node from transformed messages of its neighbors:

$$h_v^{(l+1)} = \text{UPDATE}\left(h_v^{(l)}, \text{AGGREGATE}(\{h_u^{(l)} : u \in \mathcal{N}(v)\})\right).$$

GCN, GraphSAGE, GAT differ only in how they aggregate and weight neighbors.

Axes measured. Depth sweep $\{2, 4, 8, 16, 32\}$; **10 seeds** per config, mean $\pm$ std. Predictive: **accuracy + macro-F1** (micro-F1 equals accuracy for single-label multiclass, so it is redundant). Geometry: **Dirichlet energy (per layer)** and **MAD**.

Course / group. SEP 740 / CSE 705 Deep Learning (Dr. Anwar Mirza), McMaster. Kiarash — lead & integration owner (architectures, harness, depth-sweep infra, integration). Yiheng — metrics (Dirichlet energy, MAD), visualization, hyperparameter tuning, applications/noise discussion. Both co-author analysis, report, video.

Week	Dates	Tasks	Lead
Wk 1	Jul 7–13	Data pipeline; reproduce 2-layer GCN / SAGE / GAT baselines	Kiarash
Wk 2	Jul 14–20	Depth-sweep harness; implement + validate energy and MAD; run sweep	Kiarash / Yiheng
Wk 3	Jul 21–27	Mitigation ablations; hyperparameter study; embedding visualization	Shared
Wk 4	Jul 28–Aug 3	Qualitative analysis; applications & noise; final report + presentation	Both

Final deliverables. Detailed report (7 sections) + 5-page IEEE-style article + 15–25 min recorded presentation + a clean, reproducible **module repo** with a root README. Code is AI-reviewed with zero-tolerance plagiarism, and the report needs an “explanation of the source code” section — so every line must be defensible in your own words.

2 Dependency map of the material

Stage	Focus
1	GCN fundamentals: aggregation as the fix, layer construction, normalized adjacency, depth, oversmoothing, Cora setup
2	GraphSAGE and GAT via the spectral-vs-spatial lens
3	Oversmoothing metrics: Dirichlet energy, MAD; the shared interface contract with Yiheng
4	Four mitigations: residual, PairNorm, Jumping Knowledge, GCNII
5	Experimental spec to component spec files for Claude Code handoff

Per-component workflow: clarify to (for theory) understand-the-source to propose to log the decision to implement in Claude Code to critique to hand off a spec. Nothing is “done” until it passes the **defense gate**: explain the mechanism in your own words, justify one design choice, predict/interpret one result.

3 Notation & conventions

Symbol	Meaning
$N, \mathcal{N}(v)$	number of nodes (2,708); neighbor set of v
$A, I, \tilde{A} = A + I$	adjacency; identity (self-loops); adjacency with self-loops
$\tilde{D}$	degree matrix of $\tilde{A}$, $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$
$\hat{A} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$	symmetric-normalized propagation operator ($\hat{P}$ in GCNII)
$L = I - \hat{A}$	(normalized) graph Laplacian
$H^{(l)} \in \mathbb{R}^{N \times d_l}$	node embeddings at layer l ; row $h_v^{(l)}$; $X^{(0)} = H^{(0)}$ input features
$W^{(l)}, \Theta$	learnable weight (linear map)
σ	nonlinearity (ReLU unless noted)
C	number of classes (7)
$\parallel$	vector concatenation

Code naming: functions/methods **PascalCase**, variables **camelCase**; PyG Data fields keep library names (`edge_index`, `train_mask`, `x`, `y`). Strongly typed, OOP, vectorized (no Python loops over nodes/edges). American spelling.

4 Stage 1 — GCN fundamentals

4.1 4.1 Why a plain MLP fails on a graph

An MLP on each node’s features independently ignores the graph. The naive patch (feed a node plus its neighbors into an MLP) fails on a deeper problem: **a neighbor set has no canonical ordering**, so any function of $\{h_u : u \in \mathcal{N}(v)\}$ must be **permutation-invariant** — which an MLP over concatenated neighbors is not. The structural fix is **order-independent aggregation** (sum / mean / max). Everything downstream is a choice of aggregator and transform.

Two distinct uses of “order”: node *ordering* (permutation of the neighbor set) is the invariance requirement; node *degree* (how many neighbors) is what the normalization handles. Do not conflate them.

4.2 4.2 One GCN layer = aggregate to transform to nonlinearity

$$H^{(l+1)} = \sigma(\hat{A} H^{(l)} W^{(l)}) \quad (\text{Kipf \& Welling, 2017}).$$

Right to left: **aggregate** via $\hat{A}$ (permutation-invariant mixing), apply a **shared** linear map $W^{(l)}$ (the “convolution” weight sharing — same weights for every node), then a **nonlinearity**. $\hat{A}$ carries all the structure; $W^{(l)}$ is node-agnostic.

4.3 4.3 The symmetric-normalized adjacency

$$\hat{A} = \tilde{D}^{-1/2}(A + I) \tilde{D}^{-1/2}.$$

- **Self-loops (+I)**: without them a node drops its own features from the update. - **Two-sided degree normalization**: high-degree nodes would otherwise dominate and repeated multiplication would blow up or vanish. Normalizing bounds the spectrum (eigenvalues of $L = I - \hat{A}$ in $[0, 2)$), keeping activations stable across depth. - **Symmetric form**: keeps the operator symmetric, hence real spectrum + orthogonal eigenbasis — the hook the spectral view (Stage 2) hangs on. Contrast the row-normalized $\tilde{D}^{-1}\tilde{A}$ (a pure average), which is not symmetric.

4.4 4.4 Depth = stacked layers with interleaved nonlinearities

k stacked layers give a **k -hop receptive field**; each layer has its own $W^{(l)}$ and σ . Crucial contrast with **SGC**, which precomputes $\hat{A}^k X$ once and applies a single linear classifier: SGC drops the interleaved nonlinearities, collapsing depth into a one-shot low-pass propagation. A deep GCN is *repeated nonlinear* propagation — but the aggregation part still behaves like repeated low-pass filtering, which sets up oversmoothing.

4.5 4.5 Oversmoothing and its Dirichlet-energy fingerprint

Repeated $\hat{A}$ acts as a **low-pass filter**. As depth grows, representations converge toward a degree-weighted fixed point (the dominant eigenvector direction of $\hat{A}$), washing out class-discriminative variation and collapsing accuracy. **Dirichlet energy** — how much embeddings vary across edges — **decays toward zero** as this happens. Tracked *per layer*, it localizes *where* collapse begins, not just that it did. (Formal definition: Stage 3.)

4.6 4.6 Cora: semi-supervised, transductive

- **Transductive:** the *entire* graph (all node features and edges, including test nodes) is present at training; only the test/val **labels** are withheld. Contrast the inductive setting GraphSAGE targets.
- **Semi-supervised:** only 140 of 2,708 nodes carry a training label; propagate label information across structure to the rest.
- **Masks:** boolean `train_mask` / `val_mask` / `test_mask` over all N nodes select which nodes count in the loss and each evaluation. The forward pass runs over the whole graph; the mask picks the nodes that count.

Stage 1 — defense-gate prompts

- Why is order-independent aggregation (not just “add neighbors”) *the* structural fix?
- Justify one choice in $\hat{A}$ (self-loops / normalization / symmetry) against the simpler alternative.
- Predict the accuracy curve *and* the per-layer Dirichlet-energy curve for GCN depth 2 to 32; where do they turn, and why together?
- Distinguish SGC from a deep GCN in one sentence, and why it matters for oversmoothing.

5 Stage 2 — GraphSAGE & GAT via the spectral-vs-spatial lens

5.1 5.1 The lens

Two readings of the same layer:

- **Spectral view.** GCN descends from spectral graph convolution — filtering in the eigenbasis of $L = I - \hat{A}$. $\hat{A}$ is a **first-order, localized approximation** of a spectral filter, which is *why* it behaves like a fixed low-pass filter and why depth drives oversmoothing. The spectral view explains the failure mode.
- **Spatial view.** Message passing directly in the vertex domain: aggregate over spatial neighbors, no eigendecomposition, no fixed filter. GraphSAGE and GAT are **natively spatial** — they *learn* the aggregation instead of inheriting a fixed one, and never need the spectral framing.

Organizing thesis: **GCN = fixed spectral filter; GraphSAGE = learned/sampled spatial aggregation; GAT = learned anisotropic spatial weighting.** Each relaxes something GCN holds fixed.

5.2 5.2 GraphSAGE (Hamilton et al., 2017)

PyG SAGEConv, mean aggregator:

$$h_v^{(l+1)} = W_1^{(l)} h_v^{(l)} + W_2^{(l)} \cdot \text{mean}_{u \in \mathcal{N}(v)} h_u^{(l)}.$$

The defining move is the **separable self/neighbor transform**: W_1 acts on the node's own state, W_2 on the aggregated neighbors, as *separate* matrices — unlike GCN, which folds self into the aggregation via the self-loop. Aggregator is choosable: **mean** / **max-pool** / **LSTM**. Optionally the output is ℓ_2 -normalized, $h_v^{(l+1)} \leftarrow h_v^{(l+1)} / \|h_v^{(l+1)}\|_2$.

Sampling and scale. The original method samples a fixed-size neighbor subset per layer (sample-and-aggregate), which is what makes it **inductive** and lets it train on graphs too big for full-batch — the hook for the course's large-scale-graphs discussion. *On Cora we run full-batch, so sampling is conceptual here; the separable transform is the part that matters for our comparison.*

5.3 5.3 GAT (Velickovic et al., 2018)

$$h_v^{(l+1)} = \sigma \left(\sum_{u \in \mathcal{N}(v) \cup \{v\}} \alpha_{vu} \Theta^{(l)} h_u^{(l)} \right), \quad \alpha_{vu} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^\top [\Theta h_v \parallel \Theta h_u]))}{\sum_{k \in \mathcal{N}(v) \cup \{v\}} \exp(\text{LeakyReLU}(\mathbf{a}^\top [\Theta h_v \parallel \Theta h_k]))}.$$

A node weights its neighbors **unequally** through a **learned** attention score (a single vector $\mathbf{a}$ over the concatenated transformed pair, LeakyReLU, softmax over the neighborhood). **Multi-head**: run K independent heads and **concatenate** (hidden layers) or **average** (final layer). (*PyG's GATConv computes the same score as $\mathbf{a}_s^\top \Theta_s h_v + \mathbf{a}_t^\top \Theta_t h_u$ — an algebraically equivalent split of the concatenated form.*)

5.4 5.4 Side by side

	GCN	GraphSAGE	GAT
Aggregation	fixed, degree-normalized ($\hat{A}$)	mean / max-pool / LSTM	attention-weighted sum
Neighbor weighting	fixed by degree	uniform within aggregator	learned per-edge α_{vu}
Self vs neighbor	merged (self-loop)	separated (W_1 vs W_2)	self included via self-attention
Inductive?	no	yes (sample-and- aggregate)	yes
Native view	spectral	spatial	spatial

5.5 5.5 Does relaxing GCN’s fixed filter avoid oversmoothing?

No — not on its own, and this is the point of the study. Learned attention and learned aggregation change *which* neighbors dominate, but every one of these is still a neighborhood-averaging operator, so stacking them still contracts representations toward a fixed point. The empirical record on Cora is blunt: without a mitigation, GCN, GAT/GATv2, and GraphSAGE all hold up at 2–4 layers and then **collapse by ~8 layers** to near-trivial accuracy (Cora has 7 classes, so the floor is roughly random). So architecture choice alone does **not** solve depth; that is what motivates Stage 4. The measurable question our sweep answers is whether attention *delays* onset relative to fixed aggregation, not whether it prevents collapse.

Stage 2 — defense-gate prompts

- State the spectral-vs-spatial lens in two sentences and place all three architectures on it.
- Contrast GraphSAGE’s separated self/neighbor transform with GCN’s merged self-loop — what does the separation buy?
- Explain why learned attention does not, by itself, prevent oversmoothing, and what result in our sweep would show whether it delays onset.
- Explain what makes GraphSAGE inductive and why that is irrelevant to Cora’s numbers but central to the large-scale-graphs discussion.

6 Stage 3 — Oversmoothing metrics & the interface contract (Yiheng’s component)

You integrate this; Yiheng owns the implementation and freezes the conventions.

6.1 6.1 Dirichlet energy

A smoothness functional — how much embeddings differ across connected nodes:

$$E(H) = \text{tr}(H^\top L H) = \frac{1}{2} \sum_{(u,v) \in \mathcal{E}} \left\| \frac{h_u}{\sqrt{1+d_u}} - \frac{h_v}{\sqrt{1+d_v}} \right\|_2^2,$$

with L the (normalized) Laplacian and d_v the degree. $E \rightarrow 0$ exactly when embeddings become indistinguishable across edges — the oversmoothing signature. Recorded **per layer** to localize onset. Across the oversmoothing literature (Cai & Wang 2020; Rusch et al. 2023) this is the **canonical, numerically stable** measure and is preferred over MAD when the two disagree.

Convention is a decision, not a given. Normalized vs combinatorial Laplacian, and whether the degree scaling above is applied, change the numbers. Settle this **with Yiheng** and freeze it across all runs; do not assume it from memory.

6.2 6.2 MAD — mean average distance

Mean pairwise **cosine distance** between node embeddings, ranging $[0, 1]$ (smaller = more collapsed):

$$\text{MAD} = \text{mean}_{(i,j)} (1 - \cos(h_i, h_j)).$$

A scale-invariant second view: cosine ignores magnitude, so MAD catches *directional* collapse that an energy computed on unnormalized features might mask. A refinement (MADGap) contrasts MAD over *remote* node pairs against *neighbor* pairs; whether we use global MAD or the neighbor/remote split is a convention to pin in this stage. We report both energy and MAD, treating energy as the more principled of the two.

6.3 6.3 The interface contract (the shared surface — fixed once set)

- **Models return** `Forward(x, edgeIndex) -> tuple[Tensor, list[Tensor]]`, i.e. (`logits [N, C], layerEmbeddings`) — one embedding tensor per layer. This is what lets metrics run per layer without knowing the architecture.
- **Metrics consume** `layerEmbeddings + PyG Data (edge_index)` and **produce** scalar records (energy per layer, MAD).
- **Results record** (one JSON per run), seeded now, finalized in Stage 5: `arch, depth, seed, accuracy, macroF1, dirichletPerLayer[], mad`.

Any change to this surface is a **flagged contract change** both sides update — never a silent drift.

Stage 3 — defense-gate prompts

- Why must models emit *per-layer* embeddings, not just the final one?
- Give one reason energy and MAD can disagree, and which you trust when they do.
- Name one convention in the energy definition that changes the numbers and must be frozen.

7 Stage 4 — Oversmoothing mitigations

Experimental plan (proposal): apply the **full set to GCN** across the depth range, then carry the **single most effective** mitigation to GraphSAGE and GAT to test whether the fix generalizes. The four act on different parts of the layer — know *which*.

7.1 7.1 Residual / skip connections

$$h_v^{(l+1)} = \sigma(\hat{A}H^{(l)}W^{(l)})_v + h_v^{(l)}.$$

Keeps a copy of the pre-smoothing signal so repeated averaging cannot fully erase it. It is the weakest fix here because the smoothing branch still dominates as depth grows — the residual slows collapse rather than removing its cause. Acts on **how representations combine across depth**.

7.2 7.2 PairNorm (Zhao & Akoglu, 2020)

A normalization inserted **after each convolution**: center, then rescale so the total pairwise squared distance across all nodes is held constant across layers.

$$h_v^c = h_v - \frac{1}{n} \sum_i h_i \quad (\text{center}), \quad h'_v = s \cdot \frac{h_v^c}{\sqrt{\frac{1}{n} \sum_i \|h_i^c\|_2^2}} \quad (\text{scale}),$$

which fixes total pairwise squared distance at $C = 2n^2s^2$. Because *oversmoothing is exactly the shrinking of inter-node distance*, holding that quantity constant attacks the mechanism directly. It permits within-cluster smoothing (desirable) while preserving global spread. The **PairNorm-SI** variant instead scales each node to a fixed norm s ($h'_v = s h_v^c / \|h_v^c\|_2$) and is more stable for GCN. Acts on **the scale/spread of representations**.

7.3 7.3 Jumping Knowledge (Xu et al., 2018)

Instead of using only the last (most-smoothed) layer, aggregate representations from **all** layers at the output:

$$h_v^{\text{JK}} = \text{LA}(h_v^{(1)}, \dots, h_v^{(T)}), \quad \text{LA} \in \{\text{concat}, \text{max}, \text{LSTM-attention}\}.$$

This lets each node **select its own effective receptive field** rather than being forced to the same hop count as every other node — so a node whose useful signal saturates at 2 hops is not dragged past it by a 16-layer stack. It **sidesteps** oversmoothing (reads off the good early layers) rather than preventing it. Acts on **how representations combine across depth**, per node.

7.4 7.4 GCNII (Chen et al., 2020)

Two ingredients, applied every layer, that together enable very deep GCNs:

$$H^{(l+1)} = \sigma\left((1 - \alpha) \hat{P}H^{(l)} + \alpha H^{(0)}((1 - \beta_l) I + \beta_l \Theta^{(l)})\right), \quad \beta_l = \log\left(\frac{\theta}{l} + 1\right).$$

- **Initial residual** (α): mix the raw input $H^{(0)}$ back in at *every* layer, so the representation can never stray arbitrarily far from the input features — a floor on retained information. - **Identity mapping** (β_l): bias each weight toward I , with strength $\beta_l \rightarrow 0$ as depth l grows, so deep layers act closer to identity and cannot amplify the smoothing.

Together these let GCNII run to 64+ layers without collapse — the strongest of the four, and the one most likely to be carried to GraphSAGE and GAT. Acts on **both the propagation and the transform**.

7.5 7.5 Predict before you run

Expected ordering on deep Cora (record this as a learning check before the ablation): GCNII > JK $\approx$ PairNorm > plain residual, with GCNII holding accuracy essentially flat with depth while a bare residual only postpones collapse. The published depth curves (GCN collapsing by ~ 8 layers; GCNII staying high to 32–64) support this shape — but our numbers come from our own runs, not from any external table.

Stage 4 — defense-gate prompts

- For each mitigation: what does it change, and what does that therefore prevent?
- Which two act on “combination across depth”, which on “scale”, which on “both”? Map them.
- Predict the four-way ranking on deep GCN with reasoning, before the runs.
- Justify full-set-on-GCN-then-best-elsewhere over every-mitigation-on-every-architecture.

8 Stage 5 — Experimental spec to Claude Code handoff

Turn settled decisions into one spec per component (`spec/<component>_spec.md`), following `spec_template.md` exactly. Implementation and running happen in Claude Code against these specs; this Project does not write the full implementation.

Spec sections (in order): Purpose; Approach (agreed); Interface & contract touchpoints; Implementation plan; Dependencies; Assumptions & constraints; Outputs & artifacts; Test plan; Report / novelty note; Open questions.

Spec	Owner	Covers
<code>model_spec.md</code>	Kiarash	<code>GnnModel(convType, numLayers, hiddenDim, dropout)</code> , <code>BuildConv</code> , the <code>(logits, layerEmbeddings)</code> contract
<code>metrics_spec.md</code>	Yiheng	Dirichlet energy (per layer), MAD; conventions frozen
<code>train_spec.md</code>	Kiarash	training/eval loop, Adam + weight decay, dropout, early stopping on val, seeds
<code>sweep_spec.md</code>	Kiarash	depth {2, 4, 8, 16, 32} x arch x 10 seeds; writes <code>results/*.json</code>
<code>viz_spec.md</code>	Yiheng	t-SNE / UMAP of penultimate embeddings, shallow vs deep
<code>experiments_spec.md</code>	Shared	mitigation ablations; hyperparameter study

Build order (dependency-first): data pipeline to `model` (defines contract) to `metrics` (consumes it) to `train` to `sweep` to `experiments/viz`. Each test plan replaces the old single-notebook “runs top to bottom” guarantee with concrete runnable checks (e.g. identical embeddings give energy ~0; 2-layer GCN reaches ~81% test accuracy).

Stage 5 — defense-gate prompts

- For any component, state its interface touchpoints and what would count as a contract change.
- Give one concrete smoke test per component that would catch a real bug.

9 The interface contract (shared surface)

The single most important thing to keep stable — it is what lets you and Yiheng work in parallel. **Fixed once set; any change is flagged so both sides update.**

1. **Model output.** Every architecture exposes `Forward(x: Tensor, edgeIndex: Tensor) -> tuple[Tensor, list[Tensor]]`, returning (logits [N, C], layerEmbeddings). Uniform across GCN / SAGE / GAT so the sweep and metrics never branch on architecture.
2. **Data in.** PyG Data: `x [N, 1433], edge_index [2, E], y [N]`, boolean masks (`train_mask, val_mask, test_mask`). Library field names preserved.
3. **Results record (one JSON per run).** `arch, depth, seed, accuracy, macroF1, dirichletPerLayer[], mad` (finalize in Stage 5). Written to `results/<arch>_d<depth>_s<seed>.json`.

10 Repo structure & build order

A module repo, not a single notebook. Each component defined once in its module and imported by the harness; results written to `results/*.json` for reproducible aggregation.

```
models/      # GnnModel, BuildConv (Kiarash)
train/       # training / eval loop (Kiarash)
metrics/     # Dirichlet energy, MAD (Yiheng)
viz/         # t-SNE / UMAP embedding plots (Yiheng)
experiments/ # depth sweep, mitigation ablations, hyperparam study
results/     # *.json run records (source for plots/tables)
notebooks/   # exploration + report-figure generation ONLY (never source of truth)
spec/        # <component>_spec.md decision records
DECISIONS.md # append-per-decision log
CLAUDE.md    # canonical code style + report voice (source of truth)
README       # how to get the dataset and reproduce results
```

Build order: data pipeline to models (defines contract) to metrics (consumes it) to train to experiments/sweep to viz.

11 Code style & report voice

CLAUDE.md in the repo is the **source of truth** for both code style and prose voice (the two surfaces have no live link, so a change to one must be mirrored to the other).

Code. Strongly typed, OOP, explicit over clever, vectorized (no Python loops over nodes/edges — tensor / sparse ops, reductions). **PascalCase** for functions/methods, **camelCase** for variables; PyG **Data** fields keep library names. Set random seeds. American spelling.

Report prose. The “We” rule and a banned-words list govern all report writing (the list lives in CLAUDE.md — consult it there, do not reconstruct from memory). Scope note: the research report is prose throughout, so “add no prose unless asked” does **not** apply — the required sections are the deliverable.

Authorship integrity. Anything that reads as AI-generated gets **rewritten in your own voice** so it records an understanding you actually hold — not reworded to defeat detection. The code is AI-reviewed with zero plagiarism tolerance, and the report needs an “explanation of the source code” section. A line you cannot defend is a line to understand and rewrite.

12 Review / defense gate & walk-through

Because Claude Code drafts the implementation, **understanding is secured here, not assumed.**

Defense gate (per component). No component is “done” until you can, attempt-first: (1) explain its core mechanism in your own words, (2) justify one design choice, (3) predict or interpret one result. Anything drafted that you cannot defend surfaces as a **gap to close**, not a line to wave through.

Walk-through (together, small chunks). Go through the code a few related lines at a time; for each, you explain what it does and why, attempt-first. Cover **every** line — code and prose — against the actual source, never memory.

Triage tags so you can stop early under deadline without losing high-value fixes:

- [BLOCKER] — wrong / breaks correctness or a claim; must fix.
- [POINTS] — costs grade on technical quality / novelty / reproducibility.
- [POLISH] — style, clarity, nice-to-have.

The per-stage defense-gate prompts above are your self-check set.

13 Experiment grid & what each experiment proves

Experiment	Setup	What it establishes
Baseline reproduction	each arch @ 2 layers	matches published Cora accuracy (~81–83%); harness is correct
Depth sweep	arch x depth {2, 4, 8, 16, 32} x 10 seeds	accuracy vs depth + per-layer energy/MAD locate collapse onset
Mitigation ablations	full set on GCN across depth; best carried to SAGE/GAT	which fixes restore deep performance, and whether the fix generalizes
Qualitative	t-SNE / UMAP of penultimate embeddings, shallow vs deep	visual of class clusters collapsing to a point cloud
Hyperparameter study	lr, hidden dim, dropout, weight decay, val only	fair, non-cherry-picked numbers

Scope discipline (proposal): depth capped at 32 (collapse is evident well before that on a graph as small as Cora); full mitigation set only on GCN. Keeps the config count tractable while still supporting every claim.

Reporting: accuracy + macro-F1, mean $\pm$ std over 10 seeds. Never fabricate a number — every figure comes from an actually-computed run.

14 Open questions & decisions pending

Carry into the relevant stage; resolve, then log in `DECISIONS.md`.

- [Stage 2] GraphSAGE aggregator for main runs (mean vs max-pool) and whether to ℓ_2 -normalize per layer.
- [Stage 2] GAT head count and last-layer combine (concat vs average).
- [Stage 3] Dirichlet-energy convention (normalized vs combinatorial Laplacian; degree scaling) and MAD scope (global vs neighbor/remote) — settle with Yiheng and freeze.
- [Stage 3/5] Final `results` record schema fields and file naming.
- [Stage 4] Recorded prediction of mitigation ranking on Cora (learning check before runs).
- [Stage 4] GCNII α and θ ; PairNorm s and standard vs SI.
- [Stage 5] Per-component test-plan smoke checks and pass thresholds.

15 Glossary

- **Message passing** — the AGGREGATE-then-UPDATE frame all three architectures share.
- **Receptive field** — hops of structure a node integrates; grows with depth.
- **Oversmoothing** — collapse of representations toward an indistinguishable fixed point as depth grows.
- **Dirichlet energy** — Laplacian quadratic form measuring embedding variation across edges; decays to 0 under oversmoothing; the canonical, numerically stable measure.
- **MAD** — mean pairwise cosine distance between embeddings; scale-invariant collapse measure in $[0, 1]$.
- **Anisotropic aggregation** — unequal (learned) neighbor weights, as in GAT.
- **Transductive / inductive** — whole graph present at train time, only labels withheld / must generalize to unseen nodes (GraphSAGE’s target).
- **Initial residual / identity mapping** — GCNII’s two ingredients: mix in $H^{(0)}$ each layer / bias the weight toward I with depth.
- **SGC** — Simplified GCN: precomputed $\hat{A}^k X$ + one linear classifier; the no-nonlinearity contrast to a deep GCN.

16 References

- Kipf, T. N., & Welling, M. (2017). *Semi-Supervised Classification with Graph Convolutional Networks*. ICLR.
- Hamilton, W. L., Ying, R., & Leskovec, J. (2017). *Inductive Representation Learning on Large Graphs (GraphSAGE)*. NeurIPS.
- Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., & Bengio, Y. (2018). *Graph Attention Networks*. ICLR.
- Li, Q., Han, Z., & Wu, X.-M. (2018). *Deeper Insights into Graph Convolutional Networks for Semi-Supervised Learning*. AAAI.
- Xu, K., Li, C., Tian, Y., Sonobe, T., Kawarabayashi, K., & Jegelka, S. (2018). *Representation Learning on Graphs with Jumping Knowledge Networks*. ICML.
- Zhao, L., & Akoglu, L. (2020). *PairNorm: Tackling Oversmoothing in GNNs*. ICLR.
- Chen, M., Wei, Z., Huang, Z., Ding, B., & Li, Y. (2020). *Simple and Deep Graph Convolutional Networks (GCNII)*. ICML.
- Yang, Z., Cohen, W. W., & Salakhutdinov, R. (2016). *Revisiting Semi-Supervised Learning with Graph Embeddings (Planetoid)*. ICML.
- Sen, P., et al. (2008). *Collective Classification in Network Data*. AI Magazine, 29(3).
- Cai, C., & Wang, Y. (2020). *A Note on Over-Smoothing for Graph Neural Networks*. (arXiv).
- Rusch, T. K., Bronstein, M. M., & Mishra, S. (2023). *A Survey on Oversmoothing in Graph Neural Networks*. (arXiv).
- Fey, M., & Lenssen, J. E. (2019). *Fast Graph Representation Learning with PyTorch Geometric*. ICLR Workshop.

Architecture / mitigation / metric equations in this document follow the forms given by the papers above and by the PyTorch Geometric documentation (SAGEConv, GATConv, PairNorm, GCN2Conv, JumpingKnowledge).